MÁSTER FP IA Y BIG DATA · MÓDULO 5073 · BLOQUE 4

Sesión 0

Introducción

LLMs, APIs y puesta a punto del entorno

Cámara de Madrid · Curso 2026-2027

EL BLOQUE · TOSTADERO RETIRO S.L.

Qué vamos a construir

El asistente de operaciones de una distribuidora madrileña de café para hostelería: entiende los correos de sus clientes, consulta sus datos y actúa con **supervisión humana**.

Sesión	Qué añade	Entrega	Peso
S1	Integración con LLMs	E1	10 %
S2	Salidas estructuradas y herramientas	E2	15 %
S3	Agentes I: bucle y razonamiento	E3	15 %
S4	Agentes II: memoria y contexto	E4	15 %
S5	MCP I: protocolo y servidor	E5	15 %
S6	MCP II: clientes, seguridad y proyecto integrador	E6	30 %

HERRAMIENTAS

Cómo vamos a trabajar

Python 3.11+

El lenguaje del bloque, en una carpeta con su entorno virtual.

VS Code

El editor, con las extensiones de Python y Jupyter.

Git + GitHub

Historial de cambios y repositorio de vuestro código.

GitHub Copilot

Asistente de código dentro del editor.

Clave de API

OpenRouter (recomendado), guardada en un .env.

AGENDA

Dos horas: fundamentos y entorno listo

Hora	Bloque	Formato
19:00	Presentación del bloque (10')	Charla
19:10	Fundamentos de LLMs (20')	Teoría
19:30	HTTP, APIs y JSON (15')	Teoría y demo
19:45	El Python que usa el kit, I (15')	Teoría
20:00	Descanso (30')	—
20:30	El Python que usa el kit, II (10')	Teoría
20:40	Puesta a punto guiada (40')	Práctica
21:20	Cierre y deberes (10')	Cierre

19:10 · 20 MIN

Fundamentos de LLMs

Qué hace un modelo de lenguaje y por qué a veces se equivoca

1 · QUÉ HACE UN LLM

Un LLM predice el siguiente token

El texto hasta ahora

El café de especialidad se tuesta en ...

Calcula la probabilidad de cada continuación, **elige una**, la añade y **repite**, token a token.

Siguiente token · ilustrativo

Candidato	Probabilidad
lotes	32 %
pequeñas	18 %
tambor	11 %
origen	6 %
… miles más	el resto

2 · TOKENS

El texto se trocea en tokens

Ilustrativo: cada modelo trocea distinto · en español, una palabra ≈ algo más de un token



→ [3010, 41872, 409, 23514, 1958, 427, 3412, …]

Fragmentos → números

El modelo no ve letras, sino números.

Y por qué importan

Cuántos caben en una llamada y cuánto cuestan: lo vemos en la sesión 1.

3 · ENTRENAMIENTO

Cómo se entrena, en tres pasos

1

Preentrenamiento

Predecir el siguiente token sobre enormes cantidades de texto.

2

Ajuste con instrucciones

Ejemplos de pregunta y respuesta para que siga órdenes.

3

Preferencias humanas

Se ajusta con valoraciones de personas (p. ej. RLHF) para que sea útil y seguro.

Resultado: un **modelo base** se convierte en un **asistente**.

4 · HACIA DÓNDE VAMOS

De chatbot a agente

Un agente **decide, actúa con herramientas, observa el resultado y repite**.

LLM
entiende y decide
S1

+

Herramientas
consultan y actúan
S2

+

Bucle
decide, actúa, observa
S3

+

Memoria
recuerda lo útil
S4

S5–S6 MCP: un estándar para conectar el agente con herramientas y datos.

19:30 · 15 MIN

HTTP, APIs y JSON

El idioma en el que hablaremos con los modelos

Cliente y servidor



La petición lleva

Método: GET para leer, POST para enviar

URL: la dirección del recurso

Cabeceras: metadatos, p. ej. la clave

Cuerpo: los datos, normalmente JSON

La respuesta trae

Código de estado: cómo ha ido (200, 404…)

Cabeceras: p. ej. el tipo de contenido

Cuerpo: el resultado, normalmente JSON

JSON ↔ diccionario de Python

Texto JSON · lo que llega por la red (abreviado)

```
{
  "current": {
    "time": "2026-10-06T18:00",
    "temperature_2m": 19.4
  }
}
```

En Python · un diccionario

```
import json

datos = json.loads(texto)
type(datos)  # dict

datos["current"]["temperature_2m"]
# 19.4
```

{ } → dict · [ ] → list · texto → str · número → int o float · true → True · null → None

DEMO EN DIRECTO · OPEN-METEO, SIN CLAVE

¿Qué temperatura hace en Madrid?

Desde la terminal

```
curl "https://api.open-meteo.com/v1/forecast\
?latitude=40.42&longitude=-3.70\
&current=temperature_2m"
```

La respuesta es JSON: el dato está en **current.temperature_2m**.

Las APIs de LLMs funcionan igual, pero piden **una clave en una cabecera**.

Desde Python · solo biblioteca estándar

```
import json, urllib.request

URL = ("https://api.open-meteo.com/v1/forecast"
       "?latitude=40.42&longitude=-3.70"
       "&current=temperature_2m")

with urllib.request.urlopen(URL) as r:
    datos = json.load(r)
print(datos["current"]["temperature_2m"],
      "°C en Madrid")
```

19:45 · 15 + 10 MIN

El Python que usa el kit

Lo justo para leer y modificar el código de las prácticas

1 · DATOS Y FUNCIONES

Diccionarios y funciones

Un diccionario: claves → valores, como el JSON

```
producto = {
    "sku": "CAF-001",
    "nombre": "Café de especialidad Etiopía 1 kg",
    "precio": 24.50,
    "stock": 120,
}

producto["precio"]      # 24.5
producto.get("origen")  # None
producto["origen"]      # KeyError
```

Una función que devuelve un valor, y un f-string

```
def precio_total(cantidad, precio):
    return cantidad * precio

total = precio_total(4, 24.50)

print(f"Total: {total:.2f} €")
# Total: 98.00 €
```

2 · DATOS CON FORMA

@dataclass y type hints

Una clase solo para guardar datos

```
from dataclasses import dataclass
@dataclass
class Producto:
    sku: str
    nombre: str
    precio: float
    stock: int

p = Producto("CAF-001", "Etiopía", 24.5, 120)
p.precio  # 24.5
```

En el kit · llm_client.py

```
@dataclass
class Respuesta:
    texto: str
    tokens_entrada: int
    tokens_salida: int
    segundos: float
    modelo: str
```

Los type hints (**: str**, **-> float**) documentan el tipo esperado. Python no los comprueba al ejecutar, pero el editor sí los usa.

3 · ERRORES Y CONFIGURACIÓN

try/except y variables de entorno

Leer la clave y avisar si falta

```
import os
from dotenv import load_dotenv

load_dotenv()  # lee el fichero .env

try:
    clave = os.environ["OPENROUTER_API_KEY"]
except KeyError:
    print("Falta OPENROUTER_API_KEY en .env")
    raise SystemExit(1)
```

.env · fuera de git · extracto

```
OPENROUTER_API_KEY=sk-or-...
# …las demás líneas la reutilizan
OPENAI_MODEL=openai/gpt-5.4-mini
ANTHROPIC_MODEL=anthropic/claude-haiku-4.5
GEMINI_MODEL=google/gemini-3.8-flash
GEMINI_API_KEY=   # opcional
```

load_dotenv() pasa el .env a las variables de entorno; **os.environ** las lee.

Captura **el error concreto** y da un mensaje que diga qué hacer.

20:00 · 30 MINUTOS

Descanso

A la vuelta: un poco más de Python y dejamos el entorno listo.

with y sqlite3

with abre, y al salir del bloque cierra o confirma

```
with open("notas.txt", encoding="utf-8") as f:
    texto = f.read()
# aquí el fichero ya está cerrado

import sqlite3
with sqlite3.connect("tostadero.db") as con:
    con.execute("UPDATE productos SET stock = 0"
                " WHERE sku = 'MOL-020'")
# al salir: commit (o rollback si hubo error)
```

En el kit · herramientas.py

```
def _consulta(sql, params=()):
    with sqlite3.connect(BD) as con:
        con.row_factory = sqlite3.Row
        return [dict(f) for f in
                con.execute(sql, params)]

_consulta("SELECT nombre FROM clientes"
          " WHERE id = ?", (1,))
# [{'nombre': 'Bar La Esquina'}]
```

Cada **?** se rellena con un valor de **params**; **sqlite3.Row** deja convertir cada fila en un diccionario.

5 · VARIOS FICHEROS

Módulos: un script usa otro

En la misma carpeta, se importa por el nombre

```
bloque4/
    .venv/  .env
    crear_bd.py
    herramientas.py
    s2_chat_herramientas.py

# dentro de s2_chat_herramientas.py
from herramientas import FUNCIONES
```

Si el fichero está en otra carpeta: **ModuleNotFoundError**.

Ejecutar no es lo mismo que importar

```
# crear_bd.py
def main() -> None:
    ...  # crea las tablas

if __name__ == "__main__":
    main()
```

python crear_bd.py → se ejecuta main().

import crear_bd → solo se cargan sus funciones.

6 · PARA RECONOCERLO

async y await: esperar sin bloquear

Una función async se espera con await

```
import asyncio

async def consultar(sku):
    await asyncio.sleep(1)  # como esperar a la red
    return f"{sku}: OK"

async def main():
    print(await consultar("CAF-001"))

asyncio.run(main())  # arranca la primera
```

En el kit · prueba_servidor.py (sesión 5)

```
async def main() -> None:
    async with Client(mcp) as client:
        herramientas = await client.list_tools()
        ...

asyncio.run(main())
```

await solo va dentro de una función **async def**.

Sin **await**, la función no llega a ejecutarse.

ANTES DE INSTALAR · CUANDO ALGO FALLA

Leer un traceback: de abajo arriba

Un error real, en la terminal

```
$ python s1_comparativa.py
Traceback (most recent call last):
  File "…/bloque4/s1_comparativa.py", line 4, in <module>
    from llm_client import LLM
  File "…/bloque4/llm_client.py", line 8, in <module>
    import anthropic
ModuleNotFoundError: No module named 'anthropic'
```

Aquí la causa: la terminal no tiene el entorno activo, falta **(.venv)**.

1. La **última línea**: qué error es y su mensaje.
2. Las líneas **File**, de abajo arriba: en qué fichero y línea falló, y quién la llamó.
3. ¿No lo entiendes? Busca el mensaje o pregunta a Copilot.

20:40 · 40 MIN

Puesta a punto: seguid la guía

1. Python 3.11 o superior
2. VS Code con las extensiones Python y Jupyter
3. Git y una cuenta de GitHub
4. GitHub Copilot en VS Code
5. Carpeta bloque4 con su .venv: pip install -r requirements.txt e ipykernel
6. El fichero .env a partir de .env.example
7. python crear_bd.py

Más adelante: Node.js 20+ (sesión 5), Docker opcional (sesión 6). Hoy no: está al final de la guía.

Listo si…

Al ejecutar el paso 7 aparece:

```
python crear_bd.py
Base de datos creada en
…/tostadero.db
```

¿Falla algo? Levantad la mano.

NORMAS DEL BLOQUE

Copilot sí, pero con cabeza

Úsalo para

Que te explique código y mensajes de error.

Ir más rápido en lo que ya sabes hacer.

No lo uses para

Pegar código que no entiendes.

Escribirle claves de API o datos personales.

Copilot Free tiene un **cupo mensual**: guardad para la sesión 5, o pedid el **Student Developer Pack**.

Tenéis que poder **explicar todo lo que entreguéis**, línea a línea.

21:20 · DEBERES

Antes de la sesión 1

1. Terminar la guía de instalación si no dio tiempo.
2. Hacer el notebook de repaso de Python: python-refresher.ipynb.
3. Clave de API: OpenRouter con 10 € (recomendado, no obligatorio); alternativas gratis en el paso 6 de la guía. La necesitamos desde la sesión 1.

Próxima sesión

Sesión 1 · Integración con LLMs

APIs de OpenAI, Gemini y Claude